

FASE 1: GoLite

MANUAL TECNICO

Introducción

El presente documento constituye el Manual Técnico del Interpretador GoLite, desarrollado como proyecto del Laboratorio de Organización de Lenguajes y Compiladores 1. Su propósito es describir con detalle los aspectos técnicos de la implementación, sirviendo como referencia para comprender la arquitectura interna, las decisiones de diseño y el funcionamiento de cada componente del sistema.

GoLite es un lenguaje de programación inspirado en la sintaxis de Go, diseñado con un subconjunto reducido de instrucciones que permite explorar los conceptos fundamentales de la teoría de compiladores: análisis léxico, análisis sintáctico, construcción del AST, análisis semántico e interpretación.

Este manual cubre exclusivamente la Fase 1 del proyecto, que comprende variables y tipos primitivos, operadores, sentencias if/else, bucles for, break/continue y las funciones embebidas `fmt.Println`, `strconv.Atoi`, `strconv.ParseFloat` y `reflect.TypeOf`.

- **Alcance del Documento**

El manual está dirigido a un lector técnico con conocimientos en Java, teoría de compiladores y herramientas de generación de analizadores. Cubre los siguientes temas:

- Entorno de desarrollo y herramientas utilizadas.
- Arquitectura general del sistema y estructura de paquetes.
- Descripción del analizador léxico (JFlex): expresiones regulares y tokens.
- Descripción del analizador sintáctico (CUP): gramática y construcción del AST.
- Descripción de cada clase de nodo del AST
- Funcionamiento del interprete: evaluación de expresiones y ejecución de sentencias.
- Tabla de símbolos: manejo de scopes y resolución de variables
- Manejo de errores: léxico, sintáctico y semántico.

Entorno de Desarrollo y Herramientas

A continuación, se describen todas las tecnologías y herramientas utilizadas en el desarrollo del proyecto.

- **Lenguaje de Programación: Java 25**

Todo el núcleo del interprete, la interfaz gráfica y la lógica de la tabla de símbolos están implementados en Java 25. Se aprovechan las siguientes características modernas del lenguaje:

- **Pattern marching con instanceof:** permite el dispatch del visitor en el intérprete de forma concisa (`if (node instanceof LiteralNode l) { ... }`).
- **Switch expressions con yield:** utilizados en el método `coerce()` y `evaluateUnary()` del interprete para mayor legibilidad.
- **Text blocks y métodos de String:** usados en el formateo de mensajes de error y salida de consola.

- **JFlex 1.9.1 – Analizador Léxico**

JFlex es un generador de analizadores léxicos para Java. A partir de un archivo `.flex` con expresiones regulares y acciones Java, genera una clase `Lexer.java` que implementa el método `yylex()` para obtener tokens uno a uno.

El archivo `lexer.flex` define: estados del autómata (`YYINITIAL`, `TEXT`, `SIMPLE_COMMENT`, `MULTIPLE_COMMENT`), expresiones regulares para cada categoría de token, y acciones que construyen objetos `Symbol` de CUP y registran entradas en la lista de tokens para el reporte.

- **CUP 11b – Analizador Sintáctico**

CUP (Constructor of Useful Parsers) es un generador de parsers LALR(1) para Java. A partir de un archivo `.cup` con la gramática en BNF y acciones semánticas Java, genera las clases `parser.java` y `sym.java`.

Cada producción de la gramática incluye una acción semántica (`RESULT = ...`) que instancia el nodo AST correspondiente, construyendo el árbol de abajo hacia arriba durante el análisis sintáctico.

- **Apache Maven**

El proyecto utiliza Maven como sistema de construcción. El `pom.xml` configura los plugins de JFlex y CUP para que se ejecuten automáticamente durante la fase `generate-sources` de Maven, garantizando que el `Lexer.java` y `parser.java` se regeneren antes de cada compilación.

- NetBeans 28 – IDE de Desarrollo**

El IDE NetBeans se utilizó para el desarrollo, ya que provee integración nativa con Maven, soporte para Java 25, y facilita la configuración de los plugins de JFlex y CUP como parte del ciclo de vida del proyecto.

Herramienta	Versión	Uso en el Proyecto
Java	25	Lenguaje principal del interprete y GUI
JFlex	1.9.1	Generación de analizador léxico
CUP	11b	Generación del analizador sintáctico LALR(1)
Maven	3.x	Sistema de construcción y gestión de dependencias
NetBeans	28	Entorno de desarrollo integrado
Java Swing	(JDK 25)	Biblioteca para la interfaz gráfica de usuario

Arquitectura del Sistema

El intérprete sigue el flujo clásico de un compilador/interprete de una pasada, compuesto por cinco etapas encadenadas: análisis léxico, análisis sintáctico, construcción del AST, análisis semántico e interpretación. La GUI coordina todas estas etapas al presionar el botón Ejecutar.

- Estructura de Paquetes**

El paquete raíz del proyecto será: **com.mycompany.proyecto_compi1_vj26**. Para temas de facilidad el paquete **com.mycompany.proyecto_compi1_vj26** se le dará el alias de ‘root’.

Paquete	Descripción
root	Clases generadas por JFlex (Lexer) y CUP (parser, sym). Clase Main de la Aplicación.
root.ast	Interfaz ASTNode, clase base BaseNode y el nodo inicial ProgramNode.
root.ast.expressions	Los nodos que representan a las expresiones del lenguaje.
root.ast.statements	Los nodos que representa a las declaraciones del lenguaje.

root.exceptions	Clases que heredan de RuntimeException para el control Break/Continue/Return.
root.frontend	Clases para la Interfaz Gráfica de la Aplicación.
root.models	Clases para los reportes. Enums para Datos Constantes.
root.symbols	Clases Symbol y SymbolTable con gestión de scopes anidados.
root.visitor	Interfaz Visitor.
root.visitor.interpreter	Clase InterpreterVisitor.
root.visitor.interpreter.value	Clases para modelar los diferentes Tipos de Datos.

- **Flujo de Ejecución**

Cuando el usuario presiona el botón Ejecutar en la GUI, se ejecuta la siguiente cadena.

Código fuente (.glt) -> Lexer (JFlex) -> Parser (CUP) -> AST -> Interpreter -> Consola

- El Lexer lee el código carácter a carácter y emite tokens, registrando cada uno en la lista de Token.
- El Parser consume los tokens del Lexer y, usando las acciones semánticas del .cup, construye el AST de abajo hacia arriba.
- Si hay errores léxicos o sintácticos, se detiene la interpretación y se reportan en la tabla de errores.
- El Interpreter recorre el AST de arriba hacia abajo usando el patrón Visitor, ejecutando cada nodo y gestionando la tabla de símbolos.
- Los errores semánticos (tipo incompatible, variable no declarada, etc.) se recolectan y se muestran en el reporte de errores.

Analizador Léxico (JFlex)

El archivo lexer.flex define el analizador léxico del lenguaje GoLite. Implementa la lógica de Automatic Semicolon Insertion (ASI), el reporte de tokens y la detección de errores léxicos.

- Expresiones Regulares**

A continuación, se describen todas las expresiones regulares definidas en el archivo .flex y su propósito:

Nombre	Expresión Regular	Descripción
LineTerminator	\r \n \r\n	Salto de línea en cualquier plataforma (Unix, Windows, Mac)
WhiteSpace	[\t\f]	Espacios, tabulaciones y form feed. Se ignoran completamente
InputCharacter	[^\r\n]	Cualquier carácter que no sea salto de línea. Usando en comentarios.
WholeNumber	0 [1-9][0-9]*	Literal entero: el cero exacto, o un dígito no-cero seguido de cualquier cantidad de dígitos. Evita números con cero iniciales como 007.
DecimalNumber	{WholeNumber}{Dot}[0-9]+	Literal decimal: parte entera valida, punto decimal obligatorio, seguido de uno o más dígitos. Ejemplo: 3.14, 0.5
Letter	[a-zA-Z]	Cualquier letra del alfabeto inglés, mayúscula o minúscula.
ID	_?{Letter}({Letter} _){WholeNumber}*	Identificador: inicia con letra o guion bajo, continua con letras, dígitos o guion bajo. Case sensitive.
RuneLiteral	\'([^\r\n\'\\] \\[nrt\"\\] \\u[0-9A-Fa-f]{4})\'	Literal de tipo rune: un carácter simple entre comillas simples, o una secuencia de escape (\n, \t, \r, \\", \'). También acepta escapes Unicode \uXXXX.

- Estados del Autómata**

JFlex implementa un autómata de estados finitos. El leer de GoLite define cuatro estados:

Estado	Descripción
YYINITIAL	Estado inicial y principal. Reconoce todos los tokens del lenguaje

TEXT	Activo dentro de un literal string (entre comillas dobles). Maneja secuencias de escape y detecta strings sin cerrar.
SIMPLE_COMENT	Activo después de //. Ignora todo hasta el salto de línea. Al salir puede insertar un punto y coma automático (ASI).
MULTIPLE_COMENT	Activo entre /* y */. Ignora todo el contenido. Detecta el caso de comentario sin cerrar al llegar a EOF.

- **Automatic Semicolon Insert (ASI)**

GoLite hereda de Go el mecanismo de inserción automática de punto y coma. El lexer no requiere que el programador escriba ; al final de cada línea. En su lugar, el Lexer inserta automáticamente un token PUNTO_COMA cuando:

- Se encuentra un salto de línea (LineTerminator) o fin de archivo (EOF), Y
- El ultimo token emitido fue uno de: ID, literal numérico, literal string, literal rune, true, false, return, break, continue, ++, --,),] o }.

Esto se implementa mediante el método updateASI(int type) que actualiza el flag insertSemicolon después de cada token emitido, y las reglas de LineTerminator que verifican ese flag antes de decidir si insertar el punto y coma.

Analizador Sintáctico (CUP)

El archivo parser.cup define la gramatica LALR(1) del lenguaje GoLite y las acciones semánticas que construyen el AST. CUP genera las clases parser.java (el parser) y sym.java (los identificadores numéricos de cada terminal y no-terminal).

- Declaración de Precedencias**

Las precedencias se declaran de menor a mayor. CUP usa esta información para resolver conflictos shift/reduce en la gramática de expresiones:

Home	Operadores	Asociatividad y nota
1 (menor)		Izquierda. Cortocircuito: si el operando izquierdo es true, el derecho no se evalúa.
2	&&	Izquierda. Cortocircuito: si el operando izquierdo es false, el derecho no se evalúa.
3	!	Derecha, Negación y desigualdad.
4	== !=	Izquierda, Igualdad y desigualdad.
5	< <= > >=	Izquierda, Comparaciones relacionales.
6	+ -	Izquierda. Suma y resta.
7	/ *	Izquierda. División y multiplicación
8	%	Izquierda. Modulo.
9 (mayor)	Uminus	Derecha. Negación aritmética unario. Token virtual declarado solo para precedencia.

- Construcción del AST en Acciones Semánticas**

Cada producción del parser incluye una acción semántica que instancia el nodo AST correspondiente y lo asigna a RESULT. CUP propaga estos valores de abajo hacia arriba en el árbol de derivación. El símbolo inicial inicio retorna un ProgramNode que es la raíz del AST completo.

Ejemplo de acción semántica para una declaración de variable:

```
variable ::= VAR:v ID:name tipo_dato:t definición:d
        { : RESULT = new VarDecl(name, t, d, vleft, vright); }
```

Las variables vleft y vright son generadas automáticamente por CUP y contienen la línea y columna del token VAR, usadas para el reporte de errores.

Árbol de Sintaxis Abstracta (AST)

El AST está compuesto por 30 clases nodo organizadas en el paquete com.mycompany.proyecto_compi1_vj26.ast. Todas implementan la interfaz ASTNode y extienden la clase abstracta BaseNode que provee los campos line y column.

- Jerarquía de Clases

Clase	Descripción	Imagen de Referencia a Código	
ProgramNode	Raíz del AST. Contiene las referencias a la función main, funciones de usuario y declaraciones de structs.	<pre>public class ProgramNode extends BaseNode { private final FuncDecl mainFunction; private final List<FuncDecl> userFunctions; private final List<StructDecl> structDecls; public ProgramNode(FuncDecl mainFunction, List<FuncDecl> userFunctions, int line, int column) { this(mainFunction, userFunctions, new ArrayList<>(), line, column); } public ProgramNode(FuncDecl mainFunction, List<FuncDecl> userFunctions, List<StructDecl> structDecls, int line, int column) { super(line, column); this.mainFunction = mainFunction; this.userFunctions = userFunctions; this.structDecls = structDecls; } public FuncDecl getMainFunction() { ...3 lines } public List<FuncDecl> getUserFunctions() { ...3 lines } public List<StructDecl> getStructDecls() { ...3 lines } public static class Context { public final FuncDecl mainFunction; public final List<FuncDecl> userFunctions; public final List<StructDecl> structDecls; public final int line; public final int column; public Context(ProgramNode node) { this.mainFunction = node.mainFunction; this.userFunctions = node.userFunctions; this.structDecls = node.structDecls; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>	

FuncDecl	Declaración de función. Contiene la estructura para representar cualquier tipo de función declarada en el programa.	<pre>public class FuncDecl extends BaseNode { private final String name; private final List<Object[]> params; // cada Object[] = { String nombre, VarType tipo } private final VarType returnType; // null = void (sin retorno) private final Block body; public FuncDecl(String name, List<Object[]> params, VarType returnType, Block body, int line, int column) { super(line, column); this.name = name; this.params = params; this.returnType = returnType; this.body = body; } public String getName() [...3 lines] public List<Object[]> getParams() [...3 lines] public VarType getReturnType() [...3 lines] public Block getBody() [...3 lines] public boolean isVoid() [...3 lines] public static class Context { public final String name; public final List<Object[]> params; public final VarType returnType; public final Block body; public final int line; public final int column; public Context(FuncDecl node) { this.name = node.name; this.params = node.params; this.returnType = node.returnType; this.body = node.body; this.line = node.getLine(); this.column = node.getColumn(); } public boolean isVoid() { return returnType == null; } } }</pre>
Block	Bloque {}. Contiene una List<ASTNode> de sentencias. Abre y cierra un scope.	<pre>public class Block extends BaseNode { private final List<ASTNode> statements; public Block(List<ASTNode> statements, int line, int column) { super(line, column); this.statements = statements; } public List<ASTNode> getStatements() { return statements; } public static class Context { public final List<ASTNode> statements; public final int line; public final int column; public Context(Block node) { this.statements = node.statements; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>

VarDecl	Declaración ‘var x tipo = expr’. Contiene la estructura para representar cualquier tipo de declaración para una variable.	<pre>public class VarDecl extends BaseNode { private final String name; private final VarType type; private final ASTNode initValue; //puede ser null private final String structTypeName; // null salvo type == VarType.STRUCT public VarDecl(String name, VarType type, ASTNode initValue, int line, int column) { this(name, type, initValue, null, line, column); } public VarDecl(String name, VarType type, ASTNode initValue, String structTypeName, int line, int column) { super(line, column); this.name = name; this.type = type; this.initValue = initValue; this.structTypeName = structTypeName; } public String getName() { ...3 lines } public VarType getType() { ...3 lines } public ASTNode getInitValue() { ...3 lines } public String getStructTypeName() { ...3 lines } public boolean hasInitValue() { ...3 lines } public boolean isStructType() { ...3 lines } public static class Context { public final String name; public final VarType type; public final ASTNode initValue; public final String structTypeName; public final int line; public final int column; public Context(VarDecl node) { this.name = node.name; this.type = node.type; this.initValue = node.initValue; this.structTypeName = node.structTypeName; this.line = node.getLine(); this.column = node.getColumn(); } } }</pre>
ImplicitVarDecl	Declaración ‘x := expr’. El tipo se infiere del valor en tiempo de ejecución.	<pre>public class ImplicitVarDecl extends BaseNode { private final String name; private final ASTNode value; public ImplicitVarDecl(String name, ASTNode value, int line, int column) { super(line, column); this.name = name; this.value = value; } public String getName() { return name; } public ASTNode getValue() { return value; } public static class Context { public final String name; public final ASTNode value; public final int line; public final int column; public Context(ImplicitVarDecl node) { this.name = node.name; this.value = node.value; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>

Assign	Asignación ‘x = expr’. Verifica que la variable exista y que el tipo sea compatible.	<pre>public class Assign extends BaseNode { private final String name; private final ASTNode value; public Assign(String name, ASTNode value, int line, int column) { super(line, column); this.name = name; this.value = value; } public String getName() { return name; } public ASTNode getValue() { return value; } public static class Context { public final String name; public final ASTNode value; public final int line; public final int column; public Context(Assign node) { this.name = node.name; this.value = node.value; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>
ImplicitAssign	Asignación compuesta ‘x += expr’ / ‘x -= expr’. Campo operator: "+=", "- =".	<pre>public class ImplicitAssign extends BaseNode { private final String name; private final BinaryOperator operator; // "+=" o "-=" private final ASTNode value; public ImplicitAssign(String name, BinaryOperator operator, ASTNode value, int line, int column) { super(line, column); this.name = name; this.operator = operator; this.value = value; } public String getName() { ...3 lines } public BinaryOperator getOperator() { ...3 lines } public ASTNode getValue() { ...3 lines } public static class Context { public final String name; public final BinaryOperator operator; public final ASTNode value; public final int line; public final int column; public Context(ImplicitAssign node) { this.name = node.name; this.operator = node.operator; this.value = node.value; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>

IncDec	Incremento/decremento i++ / i--. Solo valido para int y float64.	<pre>public class IncDec extends BaseNode { private final String name; private final String operator; // "++" o "--" public IncDec(String name, String operator, int line, int column) { super(line, column); this.name = name; this.operator = operator; } public String getName() { return name; } public String getOperator() { return operator; } public static class Context { public final String name; public final String operator; public final int line; public final int column; public Context(IncDec node) { this.name = node.name; this.operator = node.operator; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>
If	Representa la estructura tipica para if/else if/else.	<pre>public class If extends BaseNode { private final ASTNode condition; private final Block thenBlock; private final List<ASTNode> elseifConditions; private final List<Block> elseifBlocks; private final Block elseBlock; // null si no existe public If(ASTNode condition, Block thenBlock, List<ASTNode> elseifConditions, List<Block> elseifBlocks, Block elseBlock, int line, int column) { super(line, column); this.condition = condition; this.thenBlock = thenBlock; this.elseifConditions = elseifConditions; this.elseifBlocks = elseifBlocks; this.elseBlock = elseBlock; } public ASTNode getCondition() { ...3 lines ... } public Block getThenBlock() { ...3 lines ... } public List<ASTNode> getElseIfConditions() { ...3 lines ... } public List<Block> getElseIfBlocks() { ...3 lines ... } public Block getElseBlock() { ...3 lines ... } public boolean hasElse() { ...3 lines ... } public static class Context { public final ASTNode condition; public final Block thenBlock; public final List<ASTNode> elseifConditions; public final List<Block> elseifBlocks; public final Block elseBlock; public final int line; public final int column; public Context(If node) { this.condition = node.condition; this.thenBlock = node.thenBlock; this.elseifConditions = node.elseifConditions; this.elseifBlocks = node.elseifBlocks; this.elseBlock = node.elseBlock; this.line = node.getLine(); this.column = node.getColumn(); } } }</pre>

ForCond	for condicion { }. Representa la estructura para un ciclo for que simula un ciclo while.	<pre>public class ForCond extends BaseNode { private final ASTNode condition; private final Block body; public ForCond(ASTNode condition, Block body, int line, int column) { super(line, column); this.condition = condition; this.body = body; } public ASTNode getCondition() { return condition; } public Block getBody() { return body; } public static class Context { public final ASTNode condition; public final Block body; public final int line; public final int column; public Context(ForCond node) { this.condition = node.condition; this.body = node.body; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>
ForClassic	for init; cond; inc { }. Representa la estructura tipica de un ciclo for.	<pre>public class For extends BaseNode { private final ASTNode init; private final ASTNode condition; private final ASTNode increment; private final Block body; public For(ASTNode init, ASTNode condition, ASTNode increment, Block body, int line, int column) { super(line, column); this.init = init; this.condition = condition; this.increment = increment; this.body = body; } public ASTNode getInit() [...3 lines] public ASTNode getCondition() [...3 lines] public ASTNode getIncrement() [...3 lines] public Block getBody() [...3 lines] public static class Context { public final ASTNode init; public final ASTNode condition; public final ASTNode increment; public final Block body; public final int line; public final int column; public Context(For node) { this.init = node.init; this.condition = node.condition; this.increment = node.increment; this.body = node.body; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>

Break	Sentencia break. Solo valido dentro de un bucle. Lanza BreakException.	<pre>public class Break extends BaseNode { public Break(int line, int column) { super(line, column); } public static class Context { public final int line; public final int column; public Context(Break node) { this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>
Continue	Sentencia continue. Solo valido dentro de un bucle. Lanza ContinueException.	<pre>public class Continue extends BaseNode { public Continue(int line, int column) { super(line, column); } public static class Context { public final int line; public final int column; public Context(Continue node) { this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>

Binary	Expresión binaria. Representa una operación entre dos operandos.	<pre>public class Binary extends BaseNode { private final ASTNode left; private final BinaryOperator operator; private final ASTNode right; public Binary(ASTNode left, BinaryOperator operator, ASTNode right, int line, int column) { super(line, column); this.left = left; this.operator = operator; this.right = right; } public ASTNode getLeft() [...3 lines] public BinaryOperator getOperator() [...3 lines] public ASTNode getRight() [...3 lines] public static class Context { public final ASTNode left; public final BinaryOperator operator; public final ASTNode right; public final int line; public final int column; public Context(Binary node) { this.left = node.left; this.operator = node.operator; this.right = node.right; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>
Unary	Expresión unaria. Representa una operación para un operando.	<pre>public class Unary extends BaseNode { private final String operator; // "-" o "!" private final ASTNode operand; public Unary(String operator, ASTNode operand, int line, int column) { super(line, column); this.operator = operator; this.operand = operand; } public String getOperator() { return operator; } public ASTNode getOperand() { return operand; } public static class Context { public final String operator; public final ASTNode operand; public final int line; public final int column; public Context(Unary node) { this.operator = node.operator; this.operand = node.operand; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>

Literal	Literal primitivo. Representa un valor dentro de del programa.	<pre>public class Literal extends BaseNode { private final ValueWrapper value; public Literal(ValueWrapper value, int line, int column) { super(line, column); this.value = value; } public ValueWrapper getValue() { return value; } public static class Context { public final ValueWrapper value; public final int line; public final int column; public Context(Literal nodo) { this.value = nodo.value; this.line = nodo.getLine(); this.column = nodo.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>	
Identifier	Acceso a variable. Representa a una variable dentro del programa.	<pre>public class Identifier extends BaseNode { private final String name; public Identifier(String name, int line, int column) { super(line, column); this.name = name; } public String getName() { return name; } public static class Context { public final String name; public final int line; public final int column; public Context(Identifier node) { this.name = node.name; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>	

FuncCall	Llamada a función embebida o una función de usuario.	<pre>public class FuncCall extends BaseNode { private final FuncName functionName; private final List<ASTNode> args; public FuncCall(FuncName functionName, List<ASTNode> args, int line, int column) { super(line, column); this.functionName = functionName; this.args = args; } public FuncName getFunctionName() { return functionName; } public List<ASTNode> getArgs() { return args; } public static class Context { public final FuncName functionName; public final List<ASTNode> args; public final int line; public final int column; public Context(FuncCall node) { this.functionName = node.functionName; this.args = node.args; this.line = node.getLine(); this.column = node.getColumn(); } } @Override public <T> T accept(Visitor<T> visitor) { return visitor.visit(new Context(this)); } }</pre>
----------	--	---

Tabla de Símbolos

La tabla de símbolos esta implementada en el paquete `com.mycompany.proyecto_compi1_vj26.symbols` con dos clases: `Symbol` y `SymbolTable`.

- **Clase Symbol**

Representa una variable declarada. Sus campos son:

- `name (String)`: nombre del identificador.
- `type (VarType)`: tipo estático declarado (`int`, `float64`, `string`, `bool`, `rune`).
- `value (ValueWrapper)`: valor actual en tiempo de ejecución (`IntVale`, `DoubleValue`, `StringValue`, `BoolValue`, `CharValue`, `NullValue`).
- `scope (int)`: nivel de anidamiento en que fue declarada (0 = funcion main, 1+ = bloques anidados).
- `line / column (int)`: posición de la declaración para el reporte de errores.

```
public class Symbol {

    private final String name;
    private VarType type;
    private ValueWrapper value;
    private final int scope;
    private final int line;
    private final int column;
    private final SymbolType symbolType;

    public Symbol(String name, VarType type, ValueWrapper value, int scope, int line, int column) {
        this(name, type, value, scope, line, column, SymbolType.VARIABLES);
    }

    public Symbol(String name, VarType type, ValueWrapper value, int scope, int line, int column, SymbolType symbolType) {
        this.name = name;
        this.type = type;
        this.value = value;
        this.scope = scope;
        this.line = line;
        this.column = column;
        this.symbolType = symbolType;
    }

    public String getName() [...3 lines]

    public VarType getType() [...3 lines]

    public void setType(VarType type) [...3 lines]

    public ValueWrapper getValue() [...3 lines]

    public void setValue(ValueWrapper value) [...3 lines]

    public int getScope() [...3 lines]

    public int getLine() [...3 lines]

    public int getColumn() [...3 lines]

    public SymbolType getSymbolType() [...3 lines]

    @Override
    public String toString() {
        return "Symbol{" + "name=" + name + ", type=" + type + ", value=" + value + ", scope=" + scope + ", line=" + line + ", column=" + column + ", symbolType=" + symbolType + "}";
    }
}
```

- **Clase SymbolTable**

Implementa la pila de scopes usando una `List<LinkedHashMap<String, Symbol>>`. Los `LinkedHashMap` preservan el orden de inserción, lo que facilita la generación del reporte de tabla de símbolos.

Método	Comportamiento
<code>pushScope()</code>	Agrega un nuevo <code>LinkedHashMap</code> a la pila. Se llama al entrar a cada bloque <code>{ }</code> .

popScope()	Elimina el mapa del tope. Se llama al salir de cada bloque. Las variables locales dejan de ser visibles.
declare()	Declara una variable en el scope actual. Si ya existe en ese scope (mismo nivel), reporta error semántico.
lookup()	Busca de adentro hacia afuera. Si hay shadowing, retorna la variable del scope más interno.
assign()	Actualiza el valor en el scope donde fue declarada la variable, no en el scope actual.
reset()	Limpia toda la pila y los contadores. Se llama al inicio de cada nueva ejecución desde la GUI.

```
public class SymbolTable {

    private final List<LinkedHashMap<String, Symbol>> scopes;
    private final HashMap<String, FuncDecl> functions;
    private final HashMap<String, StructDecl> structs;

    private final List<Symbol> allDeclaredSymbols;

    private int currentLevel;
    private boolean isBlockFor;

    private final List<ErrorReport> semanticErrors;

    public SymbolTable(List<ErrorReport> semanticErrors) {
        this.scopes = new ArrayList<>();
        this.functions = new HashMap<>();
        this.structs = new HashMap<>();
        this.allDeclaredSymbols = new ArrayList<>();
        this.semanticErrors = semanticErrors;
        this.currentLevel = -1;
        this.isBlockFor = false;
    }

    public HashMap<String, FuncDecl> getFunctions() { ...3 lines }

    public HashMap<String, StructDecl> getStructs() { ...3 lines }

    public List<Symbol> getAllDeclaredSymbols() { ...3 lines }

    public int getCurrentLevel() { ...3 lines }

    public boolean isIsBlockFor() { ...3 lines }

    public void setIsBlockFor(boolean isBlockFor) { ...3 lines }

    public List<ErrorReport> getSemanticErrors() { ...3 lines }

    public void pushScope() {
        this.currentLevel++;
        this.scopes.add(new LinkedHashMap<>());
    }

    public void popScope() {
        if (!this.scopes.isEmpty()) {
            this.scopes.remove(scopes.size() - 1);
            this.currentLevel--;
        }
    }

    public boolean declare(String name, VarType type, ValueWrapper value, int line, int column) {
        LinkedHashMap<String, Symbol> currentScope = this.currentScope();

        if (currentScope.containsKey(name)) {
            if (this.isBlockFor && !currentScope.keySet().iterator().next().equals(name)) {
                currentScope.get(name).setValue(value);
                return true;
            } else {
                this.addError(
                    "La variable \"" + name + "\" ya fue declarada en este ámbito",
                    line, column
                );
                return false;
            }
        }

        Symbol sym = new Symbol(name, type, value, this.currentLevel, line, column, SymbolType.VARIABLE);
        currentScope.put(name, sym);
        this.allDeclaredSymbols.add(sym);
        return true;
    }

    public Symbol lookUp(String name, int line, int column) {
        Symbol sym = null;
        for (int i = this.scopes.size() - 1; i >= 0; i--) {
            sym = this.scopes.get(i).get(name);
            if (sym != null) {
                break;
            }
        }

        if (sym == null) {
            this.addError("La variable \"" + name + "\" no ha sido declarada", line, column);
        }

        return sym;
    }

    public boolean existsInCurrentScope(String name) {
        return this.currentScope().containsKey(name);
    }

    private LinkedHashMap<String, Symbol> currentScope() {
        if (this.scopes.isEmpty()) {
            throw new IllegalStateException("No hay ningún scope abierto.");
        }

        return this.scopes.get(this.scopes.size() - 1);
    }
}
```

Visitor (Interpreter)

La clase VisitorInterpreter en el paquete `com.mycompany.proyecto_compi1_vj26.visitor.interpreter` es el núcleo de ejecución. Implementa el patrón Visitor en Java. El método principal es:

- `visit(Context)`: visita los nodos de sentencias y expresiones retornando un `ValueWrapper` (`IntValue`, `DoubleValue`, `StringValue`, `BoolValue`, `CharValue`, `NullValue`, `VoidValue`).

```
public interface Visitor<T> {  
  
    T visit(ASTNode node);  
  
    T visit(ProgramNode.Context ctx);  
  
    T visit(Binary.Context ctx);  
  
    T visit(FieldAccess.Context ctx);  
  
    T visit(FuncCall.Context ctx);  
  
    T visit(Identifier.Context ctx);  
  
    T visit(IndexAccess.Context ctx);  
  
    T visit(Literal.Context ctx);  
  
    T visit(SliceLiteral.Context ctx);  
  
    T visit(StructLiteral.Context ctx);  
  
    T visit(Unary.Context ctx);  
  
    T visit(UserFuncCall.Context ctx);  
  
    T visit(Assign.Context ctx);  
  
    T visit(Block.Context ctx);  
  
    T visit(Break.Context ctx);  
  
    T visit(Continue.Context ctx);  
  
    T visit(FieldAssign.Context ctx);  
  
    T visit(For.Context ctx);  
  
    T visit(ForCond.Context ctx);  
  
    T visit(ForRange.Context ctx);  
  
    T visit(FuncDecl.Context ctx);  
  
    T visit(If.Context ctx);  
  
}
```

- **Reglas de Tipos en Operaciones Binarias**

Operando izquierdo	Operando derecho	Resultado
int	int	int (division entera para /)
int	float64	float64 (promoción implícita)
float64	int	float64 (promoción implícita)
float64	float64	float64
string	string	string (solo con operador +, concatenación)
rune	int	int (el rune se trata como su valor Unicode)

- **Transferencia de Control con Excepciones**

El intérprete usa tres excepciones de control que extienden RuntimeException. Se usan como mecanismo de transferencia de control, NO como errores:

- BreakException: lanzada por executeBreak(). Capturada en executeForCond() y executeForClassic() para salir del bucle.
- ContinueException: lanzada por executeContinue(). Capturada en el cuerpo del bucle para saltar al siguiente ciclo (y ejecutar el incremento en el for clásico).
- ReturnException: lanzada cuando se encuentra un nodo return. Capturada en executeFunc() para detener la ejecución de la función. Lleva un campo value con el valor de retorno (null en Fase 1).

Las tres excepciones se construyen con super(null, null, true, false), deshabilitando el llenado del stack trace para evitar overhead innecesario.

Manejo de Errores

El sistema de errores esta centralizado en el paquete `com.my.company.proyecto_compi1_vj26.models` con dos clases de datos: `Token` y `ErrorReport`.

- **Clases de Datos**
 - **Token:** almacena lexeme, type (), line, column de cada token reconocido.
 - **ErrorReport:** almacena description, line, column y type (enum: LEXICO, SINTACTICO, SEMANTICO).

- **Comportamiento por Tipo de Error**

Tipo	Origen	Comportamiento
Léxico	Character no reconocido, string sin cerrar, comentario sin cerrar.	Se registra en lexicalErrors del Lexer.
Sintáctico	Token inesperado, estructura gramatical invalida.	Se registra en syntaxErrors del parser via syntax_error(). Se intenta recuperación con la regla 'error'.
Semántico	Variable no declarada, re declaración, tipo incompatible, división por cero, break/continue fuera de bucle.	Se registra en semanticErrors del Interpreter o de SymbolTable. La ejecución continua cuando es posible.

Interfaz Gráfica (GUI)

La GUI esta implementada en el paquete `com.mycompany.proyecto_compi1_vj26.frontend` con cuatro clases Swing.

- **Descripción de Clases**

Clase	Responsabilidad	Imagen de Referencia a Código
IDEGoLite	Ventana principal. Coordina la barra de menus, el JTabbedPane, la consola y la ejecución del interprete. Maneja abrir/guardar archivos .glt.	<pre>public class IDEGoLite extends JFrame { // --- Componentes principales --- private final JTabbedPane tabbedPane; private final JTextArea console; // --- Estado de pestañas --- private final List<EditorTab> tabs = new ArrayList<>(); // --- Colores de consola --- private static final Color CONSOLE_BG = new Color(18, 18, 18); private static final Color CONSOLE_FG = new Color(200, 255, 200); private static final Color CONSOLE_ERROR = new Color(255, 100, 100); private static final Color CONSOLE_INFO = new Color(100, 180, 255); private static final String AST_OUTPUT_DIR = "ast_output"; /** * Creates new form IDEGoLite */ public IDEGoLite() { super("GoLite IDE"); setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); setSize(1000, 720); setMinimumSize(new Dimension(750, 520)); setLocationRelativeTo(null); // --- Menu --- setJMenuBar(this.buildMenuBar()); // --- Pestañas del editor --- this.tabbedPane = new JTabbedPane(JTabbedPane.TOP); this.tabbedPane.setFont(new Font("SansSerif", Font.PLAIN, 12)); this.tabbedPane.setBorder(BorderFactory.createEmptyBorder(4, 4, 0, 4)); // --- Consola --- this.console = new JTextArea(); this.console.setEditable(false); this.console.setFont(new Font("Monospaced", Font.PLAIN, 13)); this.console.setBackground(CONSOLE_BG); this.console.setForeground(CONSOLE_FG); this.console.setCaretColor(CONSOLE_FG); this.console.setMargin(new Insets(8, 10, 8, 10)); JScrollPane consoleScroll = new JScrollPane(this.console); consoleScroll.setBorder(BorderFactory.createLineBorder(new Color(60, 60, 60))); } }</pre>

```

private JMenuBar buildMenuBar() {
    JMenuBar menuBar = new JMenuBar();
    menuBar.setBackground(new Color(245, 245, 245));

    // --- Archivo ---
    JMenu menuArchivo = new JMenu("Archivo");
    this.addMenuItem(menuArchivo, "Nuevo", KeyStroke.getKeyStroke(KeyEvent.VK_N, InputEvent.CTRL_DOWN_MASK), e -> this.newTab(null, ""));
    this.addMenuItem(menuArchivo, "Abrir...", KeyStroke.getKeyStroke(KeyEvent.VK_O, InputEvent.CTRL_DOWN_MASK), e -> this.openFile());
    menuArchivo.addSeparator();
    this.addMenuItem(menuArchivo, "Guardar", KeyStroke.getKeyStroke(KeyEvent.VK_S, InputEvent.CTRL_DOWN_MASK), e -> this.saveCurrentTab());
    this.addMenuItem(menuArchivo, "Guardar como...", null, e -> this.saveCurrentTabAs());
    menuArchivo.addSeparator();
    this.addMenuItem(menuArchivo, "Salir", KeyStroke.getKeyStroke(KeyEvent.VK_C, InputEvent.CTRL_DOWN_MASK), e -> System.exit(0));

    // --- Herramientas ---
    JMenu menuHerramientas = new JMenu("Herramientas");
    this.addMenuItem(menuHerramientas, "Ejecutar", KeyStroke.getKeyStroke(KeyEvent.VK_F5, 0), e -> this.ejecutar());

    // --- Reportes ---
    JMenu menuReportes = new JMenu("Reportes");
    this.addMenuItem(menuReportes, "Tabla de Tokens", null, e -> this.mostrarUltimoReporteTokens());
    this.addMenuItem(menuReportes, "Tabla de Errores", null, e -> this.mostrarUltimoReporteErrores());
    this.addMenuItem(menuReportes, "Tabla de Símbolos", null, e -> this.mostrarUltimoReporteSimbolos());
    this.addMenuItem(menuReportes, "Árbol AST", null, e -> this.mostrarUltimoReporteAST());

    // --- Ayuda ---
    JMenu menuAyuda = new JMenu("Ayuda");
    this.addMenuItem(menuAyuda, "Acerca de...", null, e -> this.mostrarAcercaDe());

    // --- Botón Ejecutar ---
    JButton btnEjecutar = new JButton("Ejecutar");
    btnEjecutar.setFont(new Font("SansSerif", Font.BOLD, 13));
    btnEjecutar.setBackground(new Color(60, 60, 60));
    btnEjecutar.setForeground(Color.WHITE);
    btnEjecutar.setFocusPainted(false);
    btnEjecutar.setBorderPainted(false);
    btnEjecutar.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    btnEjecutar.setMargin(new Insets(4, 16, 4, 16));
    btnEjecutar.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseEntered(MouseEvent e) {
            btnEjecutar.setBackground(new Color(40, 160, 80));
        }
    });

    @Override
    public void mouseExited(MouseEvent e) {

```

```

private void addMenuItem(JMenu menu, String title, KeyStroke shortcut, ActionListener action) {
    JMenuItem item = new JMenuItem(title);
    item.setFont(new Font("SansSerif", Font.PLAIN, 12));
    if (shortcut != null) {
        item.setAccelerator(shortcut);
    }
    item.addActionListener(action);
    menu.add(item);
}

// --- Header de la consola ---
private JPanel buildConsoleHeader() {
    JPanel header = new JPanel(new BorderLayout());
    header.setBackground(new Color(45, 45, 45));
    header.setBorder(BorderFactory.createEmptyBorder(3, 10, 3, 6));

    JLabel label = new JLabel("Consola");
    label.setFont(new Font("SansSerif", Font.BOLD, 12));
    label.setForeground(new Color(200, 200, 200));

    JButton btnLimpiar = new JButton("Limpiar");
    btnLimpiar.setFont(new Font("SansSerif", Font.PLAIN, 11));
    btnLimpiar.setForeground(new Color(180, 180, 180));
    btnLimpiar.setBackground(new Color(70, 70, 70));
    btnLimpiar.setBorderPainted(false);
    btnLimpiar.setFocusPainted(false);
    btnLimpiar.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    btnLimpiar.addActionListener(e -> console.setText(""));

    header.add(label, BorderLayout.WEST);
    header.add(btnLimpiar, BorderLayout.EAST);
    return header;
}

// --- Gestión de pestañas ---
private void newTab(File file, String content) {
    EditorTab tab = new EditorTab(file, content);
    tabs.add(tab);

    int index = this.tabbedPane.getTabCount();
    this.tabbedPane.addTab(null, tab);

    TabHeader header = new TabHeader(this.tabbedPane, tab.getTabTitle(), () -> closeTab(tab));
    this.tabbedPane.setTabComponentAt(index, header);
    this.tabbedPane.setSelectedIndex(index);
}

```



```
private void closeTab(EditorTab tab) {
    int idx = tabs.indexOf(tab);
    if (idx < 0) {
        return;
    }

    if (tab.isModified()) {
        int opt = JOptionPane.showConfirmDialog(this,
            "?Guardar los cambios en \"" + tab.getTabTitle().replace(" ", "") + "\" antes de cerrar?",
            "Cambios sin guardar",
            JOptionPane.YES_NO_CANCEL_OPTION);
        if (opt == JOptionPane.CANCEL_OPTION) {
            return;
        }
        if (opt == JOptionPane.YES_OPTION) {
            saveTab(tab);
        }
    }

    tabs.remove(idx);
    this.tabbedPane.removeTabAt(idx);

    // Siempre tener al menos una pestaña abierta
    if (tabs.isEmpty()) {
        newTab(null, "");
    }
}

private void updateTabTitle(EditorTab tab, TabHeader header) {
    SwingUtilities.invokeLater(() -> header.setTitle(tab.getTabTitle()));
}

private EditorTab currentTab() {
    int idx = this.tabbedPane.getSelectedIndex();
    return (idx >= 0 && idx < tabs.size()) ? tabs.get(idx) : null;
}

// --- Archivo: Abrir / Guardar ---
private void openFile() {
    JFileChooser chooser = new JFileChooser();
    chooser.setFileFilter(new FileNameExtensionFilter("Archivos GoLite (*.glt)", "glt"));
    if (chooser.showOpenDialog(this) != JFileChooser.APPROVE_OPTION) {
        return;
    }

    File file = chooser.getSelectedFile();
}
```

```
private void saveCurrentTab() {
    EditorTab tab = this.currentTab();
    if (tab == null) {
        return;
    }
    if (tab.getFile() == null) {
        this.saveCurrentTabAs();
    } else {
        this.saveTab(tab);
    }
}

private void saveCurrentTabAs() {
    EditorTab tab = this.currentTab();
    if (tab == null) {
        return;
    }

    JFileChooser chooser = new JFileChooser();
    chooser.setFileFilter(new FileNameExtensionFilter("Archivos GoLite (*.glt)", "glt"));
    if (chooser.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) {
        return;
    }

    File file = chooser.getSelectedFile();
    if (!file.getName().endsWith(".glt")) {
        file = new File(file.getAbsolutePath() + ".glt");
    }
    tab.setFile(file);
    this.saveTab(tab);
}

private void saveTab(EditorTab tab) {
    try {
        Files.writeString(tab.getFile().toPath(), tab.getText(), StandardCharsets.UTF_8);
        tab.setModified(false);
        // Refrescar título de la pestaña
        int idx = tabs.indexOf(tab);
        if (idx >= 0) {
            Component comp = tabbedPane.getTabComponentAt(idx);
            if (comp instanceof TabHeader th) {
                th.setTitle(tab.getTabTitle());
            }
        }
        consolePrintInfo("Archivo guardado: " + tab.getFile().getName());
    } catch (IOException ex) {
    }
}
```

EditorTab

Panel de una pestaña del editor. Contiene el JTextArea principal con otro JTextArea como row header del JScrollPane. Mantiene el archivo asociado y el flag de modificado.

```
public class EditorTab extends javax.swing.JPanel {

    private static final Font EDITOR_FONT = new Font("Monospaced", Font.PLAIN, 13);

    private final JTextArea editor;
    private final JTextArea lineNumbers;
    private final JScrollPane scrollPane;
    private File file; // null si el archivo no ha sido guardado aun
    private boolean modified;

    public EditorTab(File file, String content) {
        super(new BorderLayout());
        this.file = file;
        this.modified = false;

        // --- Editor ---
        this.editor = new JTextArea(content);
        this.editor.setFont(EDITOR_FONT);
        this.editor.setTabSize(4);
        this.editor.setLineWrap(false);
        this.editor.setBackground(Color.WHITE);
        this.editor.setForeground(new Color(30, 30, 30));
        this.editor.setCaretColor(new Color(30, 30, 30));
        this.editor.setMargin(new Insets(0, 6, 0, 6));

        // --- Numeros de linea ---
        this.lineNumbers = new JTextArea("1");
        this.lineNumbers.setBackground(new Color(220, 220, 220));
        this.lineNumbers.setForeground(Color.DARK_GRAY);
        this.lineNumbers.setFont(EDITOR_FONT);
        this.lineNumbers.setEditable(false);
        this.lineNumbers.setFocusable(false);
        // Alinear el texto a la derecha con un margen
        this.lineNumbers.setMargin(new Insets(0, 5, 0, 10));

        // --- ScrollPane ---
        this.scrollPane = new JScrollPane(this.editor);

        // Usar el panel de lineas como rowHeader del scrollPane.
        // Esto sincroniza automaticamente el scroll vertical de ambos.
        this.scrollPane.setRowHeaderView(this.lineNumbers);

        // Marcar como modificado al escribir
        this.editor.getDocument().addDocumentListener(new javax.swing.event.DocumentListener() {
            @Override
            public void insertUpdate(javax.swing.event.DocumentEvent e) {
```

```
        public JTextArea getEditor() {
            return editor;
        }

        public String getText() {
            return editor.getText();
        }

        public File getFile() {
            return file;
        }

        public boolean isModified() {
            return modified;
        }

        public void setFile(File f) {
            this.file = f;
            setModified(false);
        }

        public void setModified(boolean m) {
            this.modified = m;
            // La ventana principal escucha cambios de estado a traves del DocumentListener
            // y actualiza el titulo de la pestaña cuando sea necesario.
        }

        /**
         * Nombre que debe aparecer en la pestaña. Si el archivo no tiene nombre
         * aun: "sin_titulo.glt" Si fue modificado: agrega " *" al final
         */
        @return
        */
        public String getTabTitle() {
            String name = (this.file != null) ? this.file.getName() : "sin_titulo.glt";
            return this.modified ? name + " *" : name;
        }

        /**
         * Calcula y actualiza el texto del panel de numeracion.
         */
        private void updateLineNumbers() {
            String text = editor.getText();
            if (text == null) {
                return;
            }
        }
```

TabHeader

Componente de encabezado personalizado de pestaña con JLabel del título y botón X para cerrar. El botón cambia a rojo en hover.

```
public class TabHeader extends javax.swing.JPanel {

    private final JLabel titleLabel;

    public TabHeader(JTabbedPane tabbedPane, String title, Runnable onClose) {
        super(new FlowLayout(FlowLayout.LEFT, 0, 0));
        setOpaque(false);

        // --- Título ---
        this.titleLabel = new JLabel(title);
        this.titleLabel.setFont(new Font("SansSerif", Font.PLAIN, 12));
        this.titleLabel.setBorder(BorderFactory.createEmptyBorder(0, 0, 0, 6));

        // --- Botón cerrar ---
        JButton closeBtn = new JButton("X");
        closeBtn.setPreferredSize(new Dimension(18, 18));
        closeBtn.setFont(new Font("SansSerif", Font.BOLD, 12));
        closeBtn.setMargin(new Insets(0, 0, 0, 0));
        closeBtn.setFocusPainted(false);
        closeBtn.setBorderPainted(false);
        closeBtn.setContentAreaFilled(false);
        closeBtn.setForeground(new Color(100, 100, 100));
        closeBtn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

        closeBtn.addMouseListener(new MouseAdapter() {
            @Override
            public void mouseEntered(MouseEvent e) {
                closeBtn.setForeground(new Color(200, 50, 50));
            }

            @Override
            public void mouseExited(MouseEvent e) {
                closeBtn.setForeground(new Color(100, 100, 100));
            }
        });

        closeBtn.addActionListener(e -> {
            if (onClose != null) {
                onClose.run();
            }
        });

        add(this.titleLabel);
        add(closeBtn);
    }
}
```

ReportFrame

JFrame reutilizable para reportes. Recibe el modo (TOKENS o ERRORS) y una lista de datos. Colorea filas de errores según su tipo.

```
public class ReportFrame extends javax.swing.JFrame {

    public ReportFrame(ReportType mode, List<?> data) {
        setTitle(switch (mode) {
            case TOKENS ->
                "Reporte de Tokens";
            case ERRORS ->
                "Reporte de Errores";
            case SYMBOLS ->
                "Reporte de Tabla de Simbolos";
        });
        setSize(750, 480);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JPanel main = new JPanel(new BorderLayout(8, 8));
        main.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
        main.setBackground(Color.WHITE);

        // --- Encabezado ---
        JLabel header = new JLabel(
            switch (mode) {
                case TOKENS ->
                    "Tabla de Tokens";
                case ERRORS ->
                    "Tabla de Errores";
                case SYMBOLS ->
                    "Tabla de Simbolos";
            },
            SwingConstants.CENTER
        );
        header.setFont(new Font("SansSerif", Font.BOLD, 15));
        header.setBorder(BorderFactory.createEmptyBorder(0, 0, 6, 0));
        main.add(header, BorderLayout.NORTH);

        // --- Tabla ---
        String[] columns = this.columns(mode);
        Object[][] rows = this.buildRows(mode, data);

        DefaultTableModel model = new DefaultTableModel(rows, columns) {
            @Override
            public boolean isCellEditable(int r, int c) {
                return false;
            }
        };
    }
}
```

		<pre>private String[] columns(ReportType mode) { return switch (mode) { case TOKENS -> new String[]{"No.", "Lexema", "Tipo", "Línea", "Columna"}; case ERRORS -> new String[]{"No.", "Descripción", "Línea", "Columna", "Tipo"}; case SYMBOLS -> new String[]{"No.", "ID", "Tipo símbolo", "Tipo dato", "Ámbito", "Línea", "Columna"}; }; } private Object[][] buildRows(ReportType mode, List<?> data) { Object[][] rows = new Object[data.size()][5]; for (int i = 0; i < data.size(); i++) { switch (mode) { case TOKENS -> { Token t = (Token) data.get(i); rows[i] = new Object[]{i + 1, t.getLexeme(), t.getType(), t.getLine(), t.getColumn()}; } case ERRORS -> { ErrorReport e = (ErrorReport) data.get(i); rows[i] = new Object[]{i + 1, e.getDescription(), e.getLine(), e.getColumn(), e.getType().getType()}; } case SYMBOLS -> { Symbol s = (Symbol) data.get(i); String ambito = s.getScope() <= 0 ? "Global" : "Local (nivel " + s.getScope() + ")"; rows[i] = new Object[]{i + 1, s.getName(), s.getSymbolType().getType(), s.getType().getType(), ambito, s.getLine(), s.getColumn()}; } } } return rows; } private void setColumnWidths(JTable table, ReportType mode) { int[] widths = switch (mode) { case TOKENS -> new int[]{45, 180, 180, 60, 70}; case ERRORS -> new int[]{45, 360, 60, 70, 90}; case SYMBOLS -> new int[]{45, 170, 130, 110, 130, 60, 70}; }; }</pre>
--	--	--

- Ciclo de Ejecución en la GUI**

El método ejecutar() de IDEGoLite sigue estos pasos al presionar el botón de Ejecutar o F5:

1. Obtiene el texto del EditorTab activo.
2. Instancia el Lexer con un StringReader del texto.
3. Instancia el parser pasando el Lexer. Llama a p.parse() para obtener el ProgramNode.
4. Si hay errores léxicos o sintácticos: muestra el resumen en consola, guarda los errores para el reporte.
5. Si el AST es válido: instancia el VisitorInterpreter y llama a interpreter.visit(ast).
6. Muestra la salida del interprete en la consola y registra los errores semánticos para el reporte.